



Technische Hochschule Nürnberg Georg Simon Ohm  
Fakultät Elektrotechnik Feinwerktechnik Informationstechnik

# Implementierung eines fiducialbasierten Lokalisierungssystems für Luftfahrzeuge im Innenraum

Masterarbeit im Studiengang Elektronische und Mechatronische Systeme  
Vertiefungsrichtung Automatisierungstechnik

Hannes Müller  
3796115

Betreuer: Prof. Dr. rer. nat. Pfitzner  
Betreuer: Noch nicht sicher

Abgabe: Nürnberg, 10. Juni 2026  
Sommersemester 2026

# Abstract

This thesis describes the development of a visual localization system for unmanned aerial vehicles based on fiducial markers in indoor environments. The Robot Operating System 2 is used as the middleware framework; AprilTags are used as the marker framework. The system uses the RGB images from two cameras as its input data source—one facing forward and one facing downward. The image processing and navigation calculations are performed by a mini-computer mounted on the aircraft. Position determination is based on the mapping of fiducial markers using simultaneous localization and mapping. For the extrinsic calibration of the cameras, a custom-developed algorithm for determining the respective transformation matrix is described. Position determination itself is achieved by detecting the previously mapped markers and calculating the poses of the camera lenses. The position data from both sensors are processed, fused into a single pose, and transmitted to the flight controller. The overall system enables GPS-independent positioning designed for indoor use. Finally, a prototype application is presented that can be used to control the aircraft.

# Kurzfassung

Diese Arbeit beschreibt die Entwicklung eines visuellen Lokalisierungssystems für unbemannte Luftfahrzeuge auf Basis von Fiducial-Markern im Innenraum. Als Middleware-Framework wird das Robot Operating System 2 verwendet; als Marker-Framework kommen AprilTags zum Einsatz. Das System nutzt die RGB-Bilder zweier Kameras als Eingangsdatenquelle – eine frontal und eine nach unten ausgerichtet. Die Berechnungen zur Bildverarbeitung sowie zur Navigation werden durch einen am Luftfahrzeug montierten mini-Computer durchgeführt. Grundlage der Positionsbestimmung bildet die Kartierung der Fiducial-Marker mittels Simultaner Lokalisierung und Kartierung. Zur extrinsischen Kalibrierung der Kameras wird ein eigens entwickelter Algorithmus zur Bestimmung der jeweiligen Transformationsmatrix beschrieben. Die Positionsbestimmung selbst erfolgt durch die Erkennung der zuvor kartierten Marker sowie die Berechnung der Posen der Kameraoptiken. Die Positionsdaten beider Sensoren werden aufbereitet, zu einer einzigen Pose fusioniert und an den Flugcontroller übermittelt. Das Gesamtsystem ermöglicht eine GPS-unabhängige Lokalisierung, die für den Einsatz in Innenräumen ausgelegt ist. Abschließend wird eine prototypische Anwendung vorgestellt, die zur Steuerung des Luftfahrzeugs eingesetzt werden kann.

# Inhaltsverzeichnis

|                                                                                  |     |
|----------------------------------------------------------------------------------|-----|
| Abstract .....                                                                   | III |
| Kurzfassung .....                                                                | IV  |
| Abkürzungsverzeichnis .....                                                      | VII |
| 1 Einleitung .....                                                               | 1   |
| 2 Theoretische Grundlagen .....                                                  | 2   |
| 2.1 Grundlagen Unbemannter Flugroboter .....                                     | 2   |
| 2.1.1 Mechanischer Aufbau am Beispiel der Holybro X650 .....                     | 2   |
| 2.1.2 Vergleich von Welt-, Roboter-, Kamera- und Fiducialkoordinatensystem ..... | 2   |
| 2.2 Repräsentation von Position und Orientierung .....                           | 4   |
| 2.2.1 Quaternionen .....                                                         | 4   |
| 2.2.2 Rodrigues-Parameter .....                                                  | 6   |
| 2.3 Transformation von Koordinatensystemen .....                                 | 6   |
| 2.4 Sensorsysteme .....                                                          | 7   |
| 2.5 Extended Kallman Filter .....                                                | 8   |
| 2.6 Pixhawk 6X .....                                                             | 9   |
| 2.6.1 Sensorik .....                                                             | 10  |
| 2.6.2 Schnittstellen .....                                                       | 10  |
| 2.6.3 EKF2 – Zustandsschätzung und Sensorfusion .....                            | 10  |
| 2.6.4 Echtzeitfähigkeit .....                                                    | 11  |
| 2.6.4.1 PNP-Fehler .....                                                         | 11  |
| 3 Stand der Technik .....                                                        | 12  |
| 3.1 Relevanz ROS2 in der autonomen Robotik .....                                 | 12  |
| 3.2 Relevanz PX4 .....                                                           | 12  |
| 3.3 Simultane Lokalisierung und Kartierung .....                                 | 12  |
| 3.4 Algorithmen zur Bildverarbeitung .....                                       | 12  |
| 3.4.1 Evaluationsmetriken für Objekterkennung .....                              | 12  |
| 3.4.2 YOLO .....                                                                 | 12  |
| 3.4.3 SSD .....                                                                  | 12  |
| 3.5 Datensatzkonzepte .....                                                      | 12  |
| 3.5.1 2D-Daten .....                                                             | 12  |
| 3.5.2 6D-Daten .....                                                             | 12  |
| 3.6 Algorithmen zur Bildverarbeitung .....                                       | 12  |
| 3.6.1 Clustern von Punktwolken mithilfe von DBSCAN .....                         | 12  |
| 3.6.2 Tischentfernung mithilfe von RANSAC .....                                  | 12  |

|       |                                                                                          |    |
|-------|------------------------------------------------------------------------------------------|----|
| 4     | Systembeschreibung .....                                                                 | 13 |
| 4.1   | Aufbau der Testumgebung .....                                                            | 13 |
| 4.2   | Softwarearchitektur .....                                                                | 13 |
| 4.3   | Systempipeline .....                                                                     | 13 |
| 4.3.1 | elektrischer Aufbau der Holybro X650 mit Pixhawk 6X .....                                | 13 |
| 5     | Implementierung der Echtzeiterkennung .....                                              | 14 |
| 5.1   | Datensatz- und Objektauswahl .....                                                       | 14 |
| 5.2   | Training der Modelle .....                                                               | 14 |
| 5.2.1 | Yolov8 Training .....                                                                    | 14 |
| 6     | Ansteuerung des KUKA iiwa 7 R800 .....                                                   | 15 |
| 6.1   | Kommunikationsschnittstellen .....                                                       | 15 |
| 6.2   | Greifpunktberechnung .....                                                               | 15 |
| 6.2.1 | Vortrainiertes Modell Dexnet .....                                                       | 15 |
| 6.2.2 | Segmentierung mithilfe der Bounding Boxen und dbscan .....                               | 15 |
| 6.2.3 | Segmentierungslogik je Objekt .....                                                      | 15 |
| 7     | Systemintegration .....                                                                  | 16 |
| 7.1   | Gesamtintegration von Echtzeiterkennung, Greifpunktberechnung und Robotersteuerung ..... | 16 |
| 7.2   | Entwicklung eines Debugging- und Visualisierungsinterfaces .....                         | 16 |
| 7.2.1 | Definieren von Ablageslots .....                                                         | 16 |
| 7.3   | Analyse des Echtzeitverhaltens .....                                                     | 16 |
| 8     | Evaluation des Greifens .....                                                            | 17 |
| 8.1   | Erfolgskriterien - Erfolgsquote und Fehlerrate .....                                     | 17 |
| 8.2   | Erkennungsgenauigkeit .....                                                              | 17 |
| 8.3   | Positioniergenauigkeit .....                                                             | 17 |
| 8.4   | Zeitliche Performance und Echtzeitfähigkeit .....                                        | 17 |
| 9     | Zusammenfassung und Ausblick .....                                                       | 18 |
| 10    | Anhang .....                                                                             | 24 |

# Abkürzungsverzeichnis

|        |                            |
|--------|----------------------------|
| EKF    | Extended Kalman Filter     |
| EKF2   | Extended Kalman Filter 2   |
| EMA    | Exponential Moving Average |
| FMU    | Flight Management Unit     |
| GPS    | Global Positioning System  |
| IMU    | Inertial Measurement Unit  |
| LiPo   | Lithium-Polymer-Akku       |
| PX4    | Pixhawk                    |
| RANSAC | Random Sample Consensus    |
| ROS2   | Robot Operating System 2   |
| UAV    | Unmanned Aerial Vehicle    |





# 1 Einleitung

Die vorliegende Arbeit beschäftigt sich mit der Entwicklung eines Fiducial-basierten Systems zur Positionserkennung eines Flugroboters im Innenraum. Als Grundgerüst des Systems dient eine Holybro X650 Unmanned Aerial Vehicle (UAV), welche mit einem Pixhawk 6X Flight Controller ausgestattet ist. Es wird auf die Parametrierung und Kalibrierung des Flight Controllers eingegangen, sodass die vom implementierten System bereitgestellte Position mit den übrigen Sensordaten durch einen Extended Kalman Filter fusioniert werden kann. Der Flight Controller ist seriell mit einem Jetson Orin NX verbunden, auf welchem ein Robot Operating System 2 (ROS2) Humble System läuft, das die Implementierung der Algorithmen sowie die Kommunikation zwischen den Komponenten ermöglicht. Die Kommunikation zwischen dem Jetson Orin NX und dem Pixhawk 6X wird über das MAVROS-Paket realisiert.

Als Sensoren zur Erkennung der Fiducials werden zwei Logitech-Kameras am Flugroboter angebracht und mit dem Jetson Orin NX verbunden, wobei eine Kamera nach unten und eine nach vorne ausgerichtet ist. Zur Auswahl der geeigneten Kameras werden die Logitech C920 Pro und die Logitech C922 Pro anhand definierter Bewertungskriterien verglichen, die mithilfe eines Paarweisen Vergleichs gewichtet und anschließend in einer Nutzwertanalyse bewertet werden. Aufgrund des Ergebnisses wird die Logitech C920 als nach vorne gerichtete Kamera und die Logitech C922 als nach unten gerichtete Kamera ausgewählt. Die Kameras werden mittels extrinsischer Kalibrierung geometrisch aufeinander abgestimmt.

Um eine zuverlässige Positionserkennung zu gewährleisten, werden AprilTags, ArUco-Marker und WhyCon-Marker ebenfalls anhand einer gewichteten Nutzwertanalyse verglichen. AprilTags gehen als geeignetstes Fiducial hervor. Für die Kartierung des Innenraums wird die Erkennung der AprilTags über die OpenCV-Bibliothek implementiert, während die Live-Positionierung durch die hardwarebeschleunigte NVIDIA Isaac ROS2 AprilTag-Erkennung auf dem Jetson Orin NX realisiert wird.

Die Kartierung des Innenraums erfolgt auf Basis einer Faktorgrafen-basierten Zustandsschätzung. Ein erster Ansatz der Echtzeit-Kartierung wird aufgrund der zu hohen Rechenlast verworfen. In einem zweiten Ansatz wird die Kartierung offline auf einem separaten Computer unter ROS2 Jazzy durchgeführt. Die erstellten Karten werden anhand der realen Umgebung bewertet und mit verschiedenen Parametern optimiert.

Die Positionserkennung relativ zur erstellten Karte wird ebenfalls in zwei Varianten untersucht. Zum einen die Faktorgrafen-basierte Zustandsschätzung, zum anderen die direkte geometrische Posenschätzung. Aufgrund der geringeren Rechenlast und der verfügbaren hardwarebeschleunigten NVIDIA Isaac AprilTag-Unterstützung wird die direkte geometrische Posenschätzung eingesetzt. Die erkannten Tags werden mithilfe von Exponential Moving Average (EMA)-Glättung, Medianfilterung, Random Sample Consensus (RANSAC) und Kovarianzschätzung aufbereitet und dem Flight Controller bereitgestellt.

Abschließend werden Probleme und Schwachstellen des Systems diskutiert sowie Verbesserungspotenziale aufgezeigt.

## 2 Theoretische Grundlagen

Zum besseren Verständnis der vorliegenden Arbeit werden im folgenden Kapitel die theoretischen Grundlagen erläutert. Es werden grundlegende Begriffe aus den Bereichen der Robotik, der Sensorik sowie der visuellen Navigation eingeführt. Darüber hinaus wird gesondert auf den verwendeten Flugcontroller, die zugrundeliegenden Koordinatensysteme sowie die relevanten Transformationen eingegangen.

### 2.1 Grundlagen Unbemannter Flugroboter

Unter einem unbemannten Luftfahrzeug UAV, umgangssprachlich auch als Drohne bezeichnet, versteht man ein Fluggerät, welches ohne einen menschlichen Piloten an Bord betrieben werden kann. Dabei können UAVs entweder ferngesteuert von einem menschlichen Operator oder vollständig autonom durch einen bordeigenen Computer geführt werden. Die Anfänge unbemannter Luftfahrzeuge reichen bis in die 1910er Jahre zurück, wo sie vorrangig für militärische Zwecke eingesetzt wurden. Ab den 2000er Jahren fanden UAVs zunehmend Eingang in zivile Anwendungsgebiete, wie beispielsweise die Katastrophenhilfe [1, S. 1–3].

#### 2.1.1 Mechanischer Aufbau am Beispiel der Holybro X650

**Grundrahmen** Der Grundrahmen eines UAV dient als mechanische Basis für die Montage aller weiteren Komponenten. Das Design des Rahmens beeinflusst maßgeblich die Flugeigenschaften des Geräts. Das in dieser Arbeit verwendete Modell ist als Senkrechtstarter konzipiert und basiert auf einem X-förmigen Quadcopter-Rahmen. Die wesentlichen Bestandteile des Rahmens umfassen die Ausleger, das Landegestell sowie die zentrale Montageplattform für die Elektronik. Die Holybro X650 weist eine Diagonale von 650 mm auf. Abbildung 2.2 zeigt ein Bild der Holybro X650. [2] [3]

**Antriebssystem** Das Antriebssystem besteht aus vier Elektromotoren, die jeweils an einem der Ausleger montiert sind. An den Motoren sind Propeller befestigt, welche den erforderlichen Auftrieb erzeugen.

**Flugcontroller** Der Flugcontroller bildet das zentrale Steuerungselement eines UAV. Er setzt eingehende Steuerbefehle in konkrete Ansteuersignale um, die an die Motoren weitergeleitet werden. In Kapitel ?? wird näher auf den verwendeten Flugcontroller eingegangen.

**Energieversorgung** Die Energieversorgung erfolgt über einen Lithium-Polymer-Akku (LiPo). Dieser versorgt die Steuerelektronik als auch die Motoren.

**Nutzlast** Das maximale Abfluggewicht der Holybro X650 beträgt 6,3 kg, wobei der flugfertige Grundrahmen bereits ein Eigengewicht von 4,3 kg aufweist. Daraus ergibt sich eine maximale Nutzlast von 2 kg (ohne Akku) [3].

#### 2.1.2 Vergleich von Welt-, Roboter-, Kamera- und Fiducialkoordinatensystem

Dieses Kapitel behandelt die verschiedenen Koordinatensysteme, die in dieser Arbeit relevant sind. Dabei wird zwischen dem Welt-, Roboter-, Kamera- und Fiducialkoordinatensystem unterschieden.

Weiterhin werden die grundlegenden Konzepte zur Repräsentation von Position und Orientierung im Raum erläutert.

Ein Koordinatensystem ist ein mathematisches Konstrukt, das zur eindeutigen Beschreibung der Lage von Punkten im Raum verwendet wird. Zur Beschreibung von Bewegungen ist die Festlegung eines Koordinatensystems von entscheidender Bedeutung. Es besteht aus einem Bezugspunkt, dem Ursprung, sowie drei orthogonalen Achsen, die als X-, Y- und Z-Achse bezeichnet werden. In dieser Arbeit wird standardmäßig das rechtshändige Koordinatensystem verwendet [4, S. 17]. Mathematisch wird der Zusammenhang durch

$$\mathbf{Z} = \mathbf{X} \times \mathbf{Y} \quad (2.1)$$

dargestellt. In dieser Arbeit werden die Achsen entsprechend des informellen Industriestandards wie folgt dargestellt: X-Achse in Rot, Y-Achse in Grün und Z-Achse in Blau. Ein Überblick über die in dieser Arbeit verwendeten Koordinatensysteme ist in Abbildung 2.1 dargestellt.

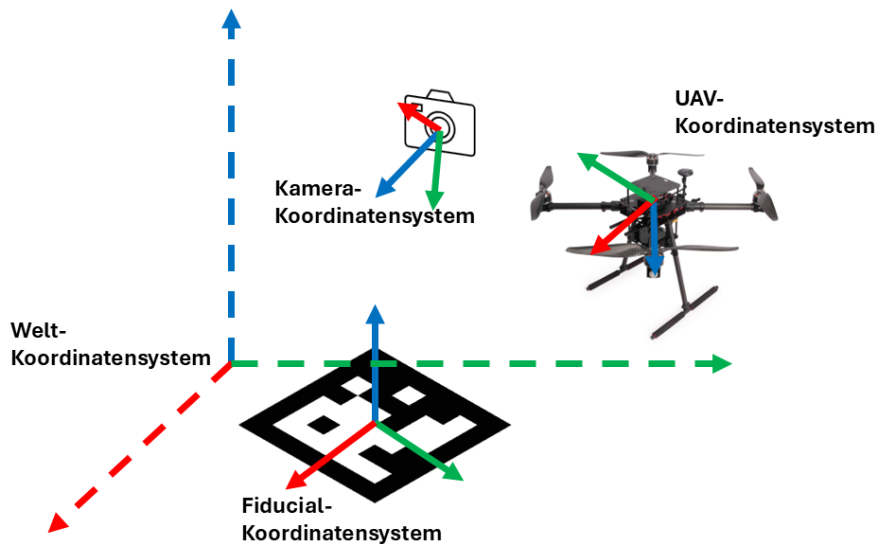


Abbildung 2.1: Vergleich der Koordinatensysteme [3]

Das Weltkoordinatensystem dient als globaler Referenzrahmen, in dem die Position aller Objekte beschrieben wird. Der Ursprung dieses Koordinatensystems kann frei gewählt werden und wird durch die Variable  $O_0$  repräsentiert. Die Achsen werden durch  $X_0$ ,  $Y_0$  und  $Z_0$  dargestellt [5, S. 4].

Das UAV-Koordinatensystem ist in der Regel vom Hersteller vorgegeben und richtet sich in diesem konkreten Fall nach dem Flugregler. In der zugehörigen Dokumentation wird angegeben, dass die X-Achse ( $X_u$ ) nach vorne, die Y-Achse ( $Y_u$ ) nach rechts und die Z-Achse ( $Z_u$ ) nach unten zeigt. Der Ursprung dieses Koordinatensystems ( $O_u$ ) befindet sich im Zentrum des Flugreglers [6].

Das Kamerakoordinatensystem ist in der Regel so definiert, dass die Z-Achse ( $Z_c$ ) entlang der optischen Achse der Kamera zeigt, die X-Achse ( $X_c$ ) nach rechts auf der Bildebene und die Y-Achse ( $Y_c$ ) nach unten auf der Bildebene zeigt. Der Ursprung ( $O_c$ ) befindet sich im optischen Zentrum der Kamera [7, S. 44].

Das Fiducialkoordinatensystem ist in dieser Arbeit so definiert, dass sich der Ursprung ( $O_f$ ) im Mittelpunkt der Fiducialmarke befindet. Die Z-Achse ( $Z_f$ ) zeigt senkrecht aus der Markerebene heraus, die X-Achse ( $X_f$ ) zeigt nach rechts in der Markerebene und die Y-Achse ( $Y_f$ ) zeigt nach oben in der Markerebene.

## 2.2 Repräsentation von Position und Orientierung

In diesem Unterkapitel wird das verwendete UAV als starrer Körper betrachtet. Die vollständige Beschreibung der Lage eines starren Körpers im Raum erfolgt durch seine Position und Orientierung [8, S. 12].

Die Position eines Körpers wird durch den Vektor

$$\mathbf{p} = (x, y, z)^T \quad (2.2)$$

beschrieben, wobei die Koordinaten einem bestimmten Koordinatensystem zugeordnet sind.

Die Orientierung eines Körpers kann auf verschiedene Weisen dargestellt werden. Im Folgenden werden die zum Verständnis dieser Arbeit relevanten Darstellungsformen erläutert. Grundsätzlich kann sich ein Körper um drei Achsen drehen, was durch die Euler-Winkel Roll ( $\phi$ ), Pitch ( $\theta$ ) und Yaw ( $\psi$ ) beschrieben wird. Abbildung 2.2 zeigt die Definition dieser Winkel am Beispiel der Holybro X650. Die Pfeilrichtung entspricht der positiven Drehrichtung im rechtshändigen Koordinatensystem.

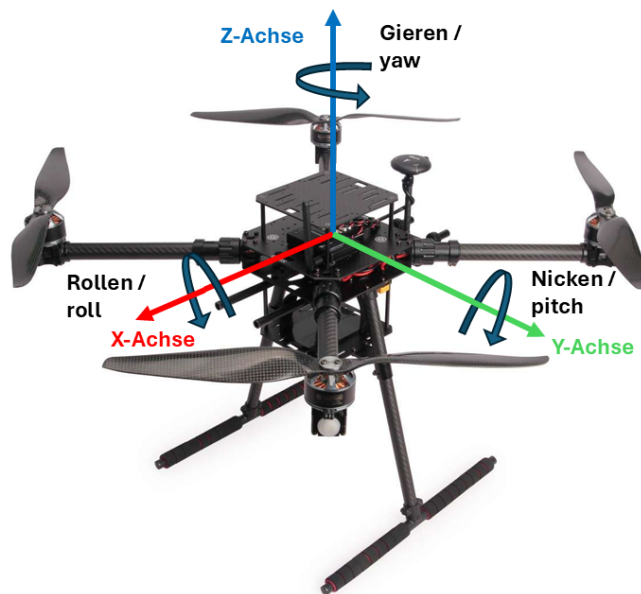


Abbildung 2.2: Roll, Pitch und Yaw am Beispiel Holybro X650 [3]

### 2.2.1 Quaternionen

Die Orientierung im Raum kann ebenfalls als Quaternion dargestellt werden. Quaternionen bieten gegenüber Euler-Winkeln den Vorteil, dass sie keine Singularitäten (Gimbal Lock) aufweisen. Qua-

ternionen stellen eine Erweiterung des komplexen Zahlenraums dar und werden durch Gleichung 2.3 definiert [8].

$$\mathbf{q} = w + ix + jy + kz \quad (2.3)$$

wobei die Komponenten  $w, x, y$  und  $z$  reelle Skalare sind und  $i, j$  sowie  $k$  die imaginären Einheiten darstellen. Die imaginären Einheiten erfüllen die folgenden Multiplikationsregeln:

$$ii = jj = kk = ijk = -1 \quad (2.4)$$

$$ij = k, \quad jk = i, \quad ki = j \quad (2.5)$$

$$ji = -k, \quad kj = -i, \quad ik = -j \quad (2.6)$$

Addition und Multiplikation von Quaternionen sind durch die Gleichungen 2.7 und 2.8 definiert.

$$q_1 + q_2 = (w_1 + w_2) + i(x_1 + x_2) + j(y_1 + y_2) + k(z_1 + z_2) \quad (2.7)$$

$$\begin{aligned} q_1 \cdot q_2 = & (w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2) \\ & + i(w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 y_2) \\ & + j(w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2) \\ & + k(w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2) \end{aligned} \quad (2.8)$$

Das Einheitsquaternion  $\mathbf{q}$  beschreibt eine Rotation um den Winkel  $\phi$  um eine normierte Rotationsachse  $\mathbf{v} = (v_x, v_y, v_z)^T$  und wird durch Gleichung 2.9 beschrieben [9].

$$\mathbf{q} = \left( \cos \frac{\phi}{2}, \mathbf{v} \cdot \sin \frac{\phi}{2} \right) = \left( \cos \frac{\phi}{2}, v_x \sin \frac{\phi}{2}, v_y \sin \frac{\phi}{2}, v_z \sin \frac{\phi}{2} \right) \quad (2.9)$$

Die Konjugation eines Quaternionen ist analog zu den komplexen Zahlen durch Gleichung 2.10 definiert.

$$\bar{\mathbf{q}} = (w, -x, -y, -z) \quad (2.10)$$

Für die Rotation eines beliebigen Punktes  $\mathbf{p} = (0, p_x, p_y, p_z)$  mittels eines Rotationsquaternionen  $\mathbf{q}$  wird Gleichung 2.11 angewendet. [10, S. 112]

$$\mathbf{p}' = \mathbf{q} \cdot \mathbf{p} \cdot \bar{\mathbf{q}} \quad (2.11)$$

Die Normalisierung eines Quaternionen erfolgt gemäß

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|}, \quad \|\mathbf{q}\| = \sqrt{x^2 + y^2 + z^2 + w^2} \quad (2.12)$$

## 2.2.2 Rodrigues-Parameter

Ein Rodrigues-Vektor ist eine alternative Möglichkeit, eine Rotation im Raum zu beschreiben. Die Rotation wird durch einen Vektor  $\mathbf{r} = (r_x, r_y, r_z)^T$  beschrieben, dessen Richtung die Rotationsachse angibt und dessen Betrag den Rotationswinkel repräsentiert [11]. Der Rotationswinkel  $\phi$  ergibt sich aus dem Betrag des Rodrigues-Vektors gemäß Gleichung 2.13 und wird gegen den Uhrzeigersinn um die Rotationsachse gemessen. [12]

$$\phi = \|\mathbf{r}\| = \sqrt{r_x^2 + r_y^2 + r_z^2} \quad (2.13)$$

Für die Umrechnung in ein Quaternion muss die Rotationsachse  $\mathbf{v}$  zunächst normiert werden:

$$\hat{\mathbf{v}} = \frac{\mathbf{r}}{\phi} = \left( \frac{r_x}{\phi}, \frac{r_y}{\phi}, \frac{r_z}{\phi} \right)^T \quad (2.14)$$

Die Umrechnung in ein Einheitsquaternion lässt sich folgendermaßen beschreiben:

$$\mathbf{q} = \left( \hat{v}_x \cdot \sin \frac{\phi}{2}, \hat{v}_y \cdot \sin \frac{\phi}{2}, \hat{v}_z \cdot \sin \frac{\phi}{2}, \cos \frac{\phi}{2} \right) \quad (2.15)$$

Eingesetzt ergibt sich:

$$\mathbf{q} = \left( \frac{r_x}{\phi} \sin \frac{\phi}{2}, \frac{r_y}{\phi} \sin \frac{\phi}{2}, \frac{r_z}{\phi} \sin \frac{\phi}{2}, \cos \frac{\phi}{2} \right) \quad (2.16)$$

Mit Hilfe der Gleichungen 2.13 und 2.16 lässt sich die Umrechnung von Rodrigues-Parametern in ein Quaternion durchführen.

## 2.3 Transformation von Koordinatensystemen

Um die genaue Position des UAVs bestimmen zu können, sind mehrere Koordinatentransformationen notwendig. Transformationen sind sowohl bei der Lokalisierung im Raum als auch bei der extrinsischen Kalibrierung der Kameras erforderlich. Es ist möglich, einen Punkt  ${}^a\mathbf{r}$ , ausgedrückt relativ zum Koordinatensystem  $a$ , relativ zum Koordinatensystem  $b$  auszudrücken, sofern Position und Orientierung des Systems  $a$  relativ zum System  $b$  bekannt sind. Ziel ist es, aus dieser Beziehung eine Transformationsmatrix abzuleiten, die zugleich die Rotation und die Translation zwischen den beiden Systemen beschreibt. [9, S. 16–17]. Die Orientierung des Systems  $a$  relativ zum System  $b$  wird durch eine Rotationsmatrix  ${}^b\mathbf{R}_a$  beschrieben. Die Lage des Ursprungs von  $a$  relativ zu  $b$  wird durch den Translationsvektor  ${}^b\mathbf{t}_a \in \mathbb{R}^3$  angegeben. Ein im System  $a$  ausgedrückter Punkt  ${}^a\mathbf{r}$  lässt sich dann mittels

$${}^b\mathbf{r} = {}^b\mathbf{R}_a {}^a\mathbf{r} + {}^b\mathbf{t}_a \quad (2.17)$$

in das System  $b$  überführen. Diese Vorschrift kombiniert eine Drehung und eine anschließende Verschiebung in einem einzigen Schritt. Durch Einführung homogener Koordinaten  ${}^a\tilde{\mathbf{r}} = \begin{bmatrix} {}^a\mathbf{r}^T & 1 \end{bmatrix}^T$  lässt sich die Drehung und Verschiebung in eine einzige Matrix überführen. Die zugehörige *homoge-*

ne Transformationsmatrix  ${}^b\mathbf{T}_a \in \text{SE}(3)$  ist definiert als

$${}^b\mathbf{T}_a = \begin{bmatrix} {}^b\mathbf{R}_a & {}^b\mathbf{t}_a \\ \mathbf{0}^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}. \quad (2.18)$$

Damit lässt sich Gleichung (2.17) kompakt schreiben als

$${}^b\tilde{\mathbf{r}} = {}^b\mathbf{T}_a {}^a\tilde{\mathbf{r}}. \quad (2.19)$$

Die Rotationsmatrix  ${}^b\mathbf{R}_a$  kann dabei aus einem Einheitsquaternion  $\mathbf{q} = (w, x, y, z)$  berechnet werden:

$$R(\mathbf{q}) = \begin{bmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2zw & 2xz + 2yw \\ 2xy + 2zw & 1 - 2x^2 - 2z^2 & 2yz - 2xw \\ 2xz - 2yw & 2yz + 2xw & 1 - 2x^2 - 2y^2 \end{bmatrix}. \quad (2.20)$$

Eine wesentliche Eigenschaft homogener Transformationsmatrizen ist ihre Verkettbarkeit. Sind die Transformationen  ${}^c\mathbf{T}_b$  (von  $b$  nach  $c$ ) und  ${}^b\mathbf{T}_a$  (von  $a$  nach  $b$ ) bekannt, so ergibt sich die direkte Transformation von  $a$  nach  $c$  durch

$${}^c\mathbf{T}_a = {}^c\mathbf{T}_b {}^b\mathbf{T}_a. \quad (2.21)$$

## 2.4 Sensorsysteme

Zum besseren Verständnis der Arbeit werden in diesem Unterkapitel die relevanten Sensortypen vorgestellt.

Die zentrale Sensoreinheit für ein UAV ist die Inertial Measurement Unit (IMU), welche Beschleunigungen und Winkelgeschwindigkeiten misst. Die IMU besteht typischerweise aus einem 3-Achsen-Beschleunigungssensor und einem 3-Achsen-Gyroskop. In einigen Fällen kann auch ein 3-Achsen-Magnetometer integriert sein, um die Orientierung relativ zum Erdmagnetfeld zu bestimmen. Ein weiterer Sensortyp ist das Barometer zur Bestimmung der Höhe, indem es den aktuellen Luftdruck misst. Weiterhin kann auch die Motordrehzahl oder Motorlast als Sensorinformation genutzt werden, um Rückschlüsse auf den aktuellen Bewegungszustand zu ziehen [8]. All diese Sensoren liefern nicht direkt die Position oder Orientierung des Roboters, reagieren jedoch schnell auf Bewegungsänderungen. Jeder dieser Sensoren ist mit einer gewissen Messunsicherheit behaftet, die durch Rauschen, Kalibrierungsfehler oder Umwelteinflüsse unterschiedlich stark ausgeprägt sein kann. Diese Messunsicherheiten lassen sich grundsätzlich in deterministische und stochastische Fehleranteile unterteilen [13]. Ein wesentlicher deterministischer Fehler ist der Bias, ein systematischer Messfehler in Form eines konstanten Versatzes, der die Messung auch dann verfälscht, wenn keine physikalische Größe anliegt. So gibt beispielsweise ein ruhendes Gyroskop trotz fehlender Drehung einen von null verschiedenen Wert aus. Dem gegenüber steht das Rauschen als stochastischer, mittelwertfreier Fehleranteil. Die Sensorfusion, also die Kombination der Daten mehrerer Sensoren, ermöglicht es, die Genauigkeit der Messungen zu verbessern [14].

Die Integration dieser Sensordaten über die Zeit ermöglicht es, die Bewegung des Roboters zu verfolgen und eine Schätzung seines Zustands zu erhalten, was als Odometrie bezeichnet wird. Da die

Sensoren, wie zuvor beschrieben, trotz Fusion fehlerbehaftet sind, führt die Integration über die Zeit zu einer zunehmenden Unsicherheit. Die Odometrie eines Flugroboters ist besonders herausfordernd, da sich das UAV in drei Dimensionen frei bewegt. Der zunehmende Fehler der Odometrie kann durch die Verwendung von externen Referenzen, wie Global Positioning System (GPS), korrigiert werden.

## 2.5 Extended Kallman Filter

In dieser Arbeit wird nicht näher auf die Sensorfusion eingegangen, da der Extended Kalman Filter (EKF) bereits vollständig im Flugcontroller implementiert ist. Auch die speziell im Pixhawk (PX4) zugrundeliegenden mathematischen Modelle werden nicht weiter erläutert. Zum Verständnis der Positionsbestimmung sowie der Parametrierung des EKF ist es jedoch notwendig, die Grundlagen des Kalman-Filters zu kennen.

Ein Kalman-Filter ist ein Algorithmus zur Schätzung des Zustands eines dynamischen Systems durch die Kombination von Vorhersagen aus einem mathematischen Modell und Messungen von Sensoren [15]. Der EKF ist ein rekursiver Filter, wodurch er kontinuierlich neue Messwerte verarbeiten kann. Im Folgenden wird der grundlegende Aufbau des Filters geschildert. Der Filter besteht im Wesentlichen aus drei Phasen. Zu Beginn muss eine Anfangsbedingung festgelegt werden. Hierfür benötigt das System einen initialen Zustandsschätzwert  $\hat{\mathbf{x}}_0^+$  sowie die zugehörige Unsicherheit in Form der Zustandskovarianzmatrix  $\mathbf{P}_0^+$  [16]. In der Prädiktionsphase wird der Zustand des Systems zum nächsten Zeitpunkt auf Basis eines mathematischen Modells berechnet. Der Zustand  $\hat{\mathbf{x}}_k^-$  wird dabei aus dem vorherigen Zustand  $\hat{\mathbf{x}}_{k-1}^+$  und der Steuereingabe  $\mathbf{u}_{k-1}$  über die nichtlineare Modellfunktion  $f$  prädiziert:

$$\hat{\mathbf{x}}_k^- = f\left(\hat{\mathbf{x}}_{k-1}^+, \mathbf{u}_{k-1}\right) \quad (2.22)$$

Gleichzeitig wird die Zustandskovarianzmatrix  $\mathbf{P}_k^-$  fortgeschrieben, die die Unsicherheit der Schätzung beschreibt. Sie wächst in dieser Phase durch das Prozessrauschen  $\mathbf{Q}_{k-1}$  an:

$$\mathbf{P}_k^- = \mathbf{F}_{k-1} \mathbf{P}_{k-1}^+ \mathbf{F}_{k-1}^\top + \mathbf{Q}_{k-1} \quad (2.23)$$

Dabei ist  $\mathbf{F}_{k-1}$  die Jacobi-Matrix der Modellfunktion  $f$ , die durch Linearisierung um den aktuellen Schätzwert gebildet wird. Das Prozessrauschen  $\mathbf{Q}$  beschreibt, wie stark das Systemmodell vom realen Verhalten abweicht: Ein hoher Wert bedeutet ein geringeres Vertrauen in das Modell.

In der Korrekturphase wird der prädizierte Zustand auf Basis eines Messwerts  $\mathbf{z}_k$  korrigiert. Zunächst wird der Kalman-Gain  $\mathbf{K}_k$  berechnet, der bestimmt, wie stark die Messung in die Schätzung einfließt:

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^\top \left( \mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top + \mathbf{R}_k \right)^{-1} \quad (2.24)$$

Hierbei ist  $\mathbf{H}_k$  die Jacobi-Matrix der Messfunktion  $h$  und  $\mathbf{R}_k$  die Messrauschkovarianzmatrix.  $\mathbf{R}$  beschreibt die Unsicherheit der Sensormessungen: Ein hoher Wert führt zu einem geringeren Vertrauen in die Messung und damit zu einer schwächeren Korrektur. Der Kalman-Gain gewichtet folglich Modellvorhersage und Messung anhand ihrer jeweiligen Unsicherheiten.



Anschließend wird der korrigierte Zustand  $\hat{\mathbf{x}}_k^+$  aus dem prädizierten Zustand und der gewichteten Differenz zwischen tatsächlicher Messung  $\mathbf{z}_k$  und erwarteter Messung  $h(\hat{\mathbf{x}}_k^-)$  – der sogenannten Innovation – berechnet:

$$\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \left( \mathbf{z}_k - h(\hat{\mathbf{x}}_k^-) \right) \quad (2.25)$$

Abschließend wird die Zustandskovarianzmatrix aktualisiert und durch die Einbeziehung der Messung reduziert:

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- \quad (2.26)$$

Die aktualisierten Größen  $\hat{\mathbf{x}}_k^+$  und  $\mathbf{P}_k^+$  dienen im nächsten Zeitschritt wiederum als Ausgangswerte der Prädiktionsphase, wodurch sich der rekursive Charakter des Filters ergibt [16].

## 2.6 Pixhawk 6X

PX4 ist ein Open-Source-Autopilot-Flugstack, der die Flugsteuerung, Zustandsschätzung und Kommunikation eines UAV übernimmt. Das Pixhawk-Projekt wurde ursprünglich als studentisches Projekt an der ETH Zürich gestartet [17] und hat sich seitdem zu einem weit verbreiteten Open-Source-Standard in der UAV-Forschung und -Industrie entwickelt. Der in dieser Arbeit verwendete Pixhawk 6X basiert auf dem Pixhawk FMUv6X-Standard und wurde von Holybro in Zusammenarbeit mit dem PX4-Entwicklerteam entwickelt.[18] Er bildet das zentrale Steuerungselement des verwendeten UAV.

Der Flugcontroller besteht aus zwei Teilen: dem Baseboard V2A und dem FMU-6X-Modul. Das Flight Management Unit (FMU)-Modul wird auf das Baseboard aufgesteckt und verschraubt. Die Komponenten sind in Abbildung 2.3 dargestellt. Im weiteren Verlauf der Arbeit wird dieser Zusammenschluss als Flugcontroller bezeichnet. Das FMU stellt die zentrale Recheneinheit des Flugcontrollers dar, auf der der Hauptprozessor (STM32H753) sowie die PX4-Firmware betrieben werden. Das Baseboard stellt vor allem die Schnittstellen zu Sensoren, anderen Komponenten und der Stromversorgung bereit.

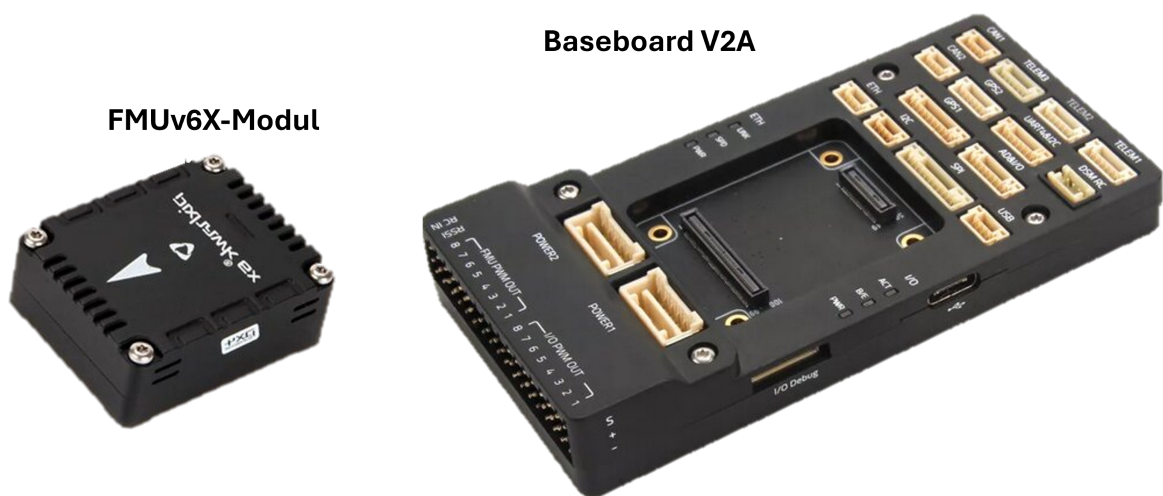


Abbildung 2.3: Komponenten des Pixhawk 6X: FMU-Modul und Baseboard V2A [3]

### 2.6.1 Sensorik

Der Pixhawk 6X verfügt über eine dreifach redundante IMU sowie zwei redundante Barometer, die auf getrennten Bussen mit unabhängiger Stromversorgung betrieben werden. Diese Redundanz erhöht die Zuverlässigkeit: Erkennt der PX4 den Ausfall eines Sensors, schaltet das System auf einen funktionierenden Sensor um, um die Flugsteuerung aufrechtzuerhalten.[18] Die Sensoren sind, wie in Abschnitt ?? beschrieben, mit einer Messunsicherheit behaftet, weshalb die redundante Auslegung und die anschließende Sensorfusion im EKF erfolgen.

Die in der eingesetzten Revision verbauten Sensoren sind in Tabelle 2.1 aufgeführt.

Tabelle 2.1: Sensorbestückung des Pixhawk 6X [18]

| Sensortyp               | Bauteil            |
|-------------------------|--------------------|
| Beschleunigung/Gyroskop | ICM-20649 / BMI088 |
| Beschleunigung/Gyroskop | ICM-42688-P        |
| Beschleunigung/Gyroskop | ICM-42670-P        |
| Barometer               | 2× BMP388          |
| Magnetometer            | BMM150             |

Zusätzlich sind die IMUs vibrationsisoliert gelagert und werden über Heizwiderstände auf einer konstanten Betriebstemperatur gehalten, um temperaturbedingte Messabweichungen zu reduzieren.[18] Ein Magnetometer ist zwar verbaut, wird in dieser Arbeit jedoch nicht zur Lagebestimmung verwendet (siehe Abschnitt 2.6.3).

### 2.6.2 Schnittstellen

Der Flugcontroller stellt über das Baseboard eine Vielzahl von Schnittstellen zur Anbindung externer Komponenten bereit. Für diese Arbeit sind insbesondere die folgenden Schnittstellen relevant:

Die serielle Schnittstelle (UART) der Telemetrie-Ports dient der Kommunikation zwischen dem Flugcontroller und dem Jetson Orin NX, über welche die extern berechnete Positionsschätzung an den EKF übermittelt wird. Die genaue Anbindung wird in Kapitel ?? beschrieben. Über den USB-Anschluss erfolgt die Parametrierung und Kalibrierung des Flugcontrollers mittels der Bodenstationssoftware QGroundControl. Die PWM-Ausgänge des Baseboards steuern die Elektromotoren des UAV an. Darüber hinaus stehen weitere Schnittstellen wie CAN, I<sup>2</sup>C und SPI zur Anbindung zusätzlicher Sensoren und Peripherie zur Verfügung, die in dieser Arbeit jedoch nicht verwendet werden.[18]

### 2.6.3 EKF2 – Zustandsschätzung und Sensorfusion

Die Zustandsschätzung des PX4 erfolgt über den Extended Kalman Filter 2 (EKF2), dieser ist in ?? beschriebenen EKF. Der EKF2 fusioniert die Messungen der bordeigenen Sensoren mit optionalen externen Referenzen, um Lage, Geschwindigkeit und Position des UAV zu schätzen. Der Zustandsvektor des EKF2 umfasst 24 Zustände, darunter die Lage als Quaternion, Geschwindigkeit, Position, die Bias-Werte von Gyroskop und Beschleunigungssensor, das Erdmagnetfeld, den Magnetometer-Bias sowie die horizontale Windgeschwindigkeit.[19] Da in dieser Arbeit kein GPS zur Verfügung steht, wird die Position des UAV über eine externe visuelle Positionsschätzung bereitgestellt. Diese

wird als externe Referenz in den EKF2 eingespeist und ersetzt dort die Funktion, die andernfalls vom GPS übernommen würde. Das Verhalten des EKF2 lässt sich über eine Reihe von Parametern konfigurieren, welche unter anderem festlegen, welche Sensorquellen zur Positions- und Höhenbestimmung herangezogen werden. Die konkrete Parametrierung für den GPS-freien Betrieb mit externer Positionsreferenz wird in Kapitel ?? erläutert.

## **2.6.4 Echtzeitfähigkeit**

### **2.6.4.1 PNP-Fehler**

## 3 Stand der Technik

Merker

"Das vorgestellte Lokalisierungssystem ist nicht hart echtzeitfähig, da es auf einem Standard-Linux-Kernel mit Python-basierten ROS2-Nodes aufbaut, die keine deterministischen Antwortzeiten garantieren. Diese Anforderung besteht jedoch auch nicht: Die hart echtzeitfähige Flugregelung erfolgt auf dem Flight Controller (PX4/NuttX), während das Jetson-System die Vision-Pose als weich echtzeitfähige Stützinformation an die Sensorfusion (EKF2) liefert. Der EKF2 überbrückt kurzzeitige Ausfälle der Vision-Pose durch IMU-gestützte Prädiktion (Inertial Dead Reckoning), wodurch gelegentliche Latenzschwankungen der Vision-Pipeline toleriert werden."

### 3.1 Relevanz ROS2 in der autonomen Robotik

### 3.2 Relevanz PX4

### 3.3 Simultane Lokalisierung und Kartierung

hier einfach schreiben

### 3.4 Algorithmen zur Bildverarbeitung

#### 3.4.1 Evaluationsmetriken für Objekterkennung

hier einfach schreiben

#### 3.4.2 YOLO

#### 3.4.3 SSD

### 3.5 Datensatzkonzepte

#### 3.5.1 2D-Daten

#### 3.5.2 6D-Daten

### 3.6 Algorithmen zur Bildverarbeitung

#### 3.6.1 Clustern von Punktwolken mithilfe von DBSCAN

#### 3.6.2 Tischentfernung mithilfe von RANSAC

## 4 Systembeschreibung

In diesem Kapitel wird die Testumgebung beschrieben, sowie auf die Softwarearchitekturen eingegangen. Außerdem wird die Systempipeline erläutert, die zur Entwicklung der Echtzeiterkennung verwendet wurde. Es werden die verschiedenen Konzepte vorgestellt, die zur Auswahl eines Echtzeitmodells geführt haben.

### 4.1 Aufbau der Testumgebung

### 4.2 Softwarearchitektur

### 4.3 Systempipeline

#### 4.3.1 elektrischer Aufbau der Holybro X650 mit Pixhawk 6X

# 5 Implementierung der Echtzeiterkennung

## 5.1 Datensatz- und Objektauswahl

## 5.2 Training der Modelle

### 5.2.1 Yolov8 Training

# **6 Ansteuerung des KUKA iiwa 7 R800**

## **6.1 Kommunikationsschnittstellen**

## **6.2 Greifpunktberechnung**

### **6.2.1 Vortrainiertes Modell Dexnet**

### **6.2.2 Segmentierung mithilfe der Bounding Boxen und dbscan**

### **6.2.3 Segmentierungslogik je Objekt**

# **7 Systemintegration**

## **7.1 Gesamtintegration von Echtzeiterkennung, Greifpunktberechnung und Robotersteuerung**

## **7.2 Entwicklung eines Debugging- und Visualisierungsinterfaces**

### **7.2.1 Definieren von Ablageslots**

## **7.3 Analyse des Echtzeitverhaltens**



## **8 Evaluation des Greifens**

**8.1 Erfolgskriterien - Erfolgsquote und Fehlerrate**

**8.2 Erkennungsgenauigkeit**

**8.3 Positioniergenauigkeit**

**8.4 Zeitliche Performance und Echtzeitfähigkeit**

## 9 Zusammenfassung und Ausblick

# Literatur

- [1] H. Studiawan und K.-K. R. Choo, *Drone and UAV Forensics: A Hands-On Approach*. Springer Nature Switzerland, 2026, ISBN: 9783031935114. DOI: 10.1007/978-3-031-93511-4.
- [2] Y.-T. Wu, Z. Qin, A. Eizad und S.-K. Lyu, "Numerical investigation of the mechanical component design of a hexacopter drone for real-time fine dust monitoring", *Journal of Mechanical Science and Technology*, Jg. 35, Nr. 7, S. 3101–3111, 2021, ISSN: 1976-3824. DOI: 10.1007/s12206-021-0632-y.
- [3] P. Modellbau, *Holybro X650*, <https://www.premium-modellbau.de/holybro-holybro-x650-development-kit-pixhawk-6c-m10-gps-433-mhz-30180>, Zugriff am 30.05.2026, 2026.
- [4] ISO, "Road Vehicles – Vehicle Dynamics and Road-Holding Ability – Vocabulary", International Organization for Standardization, Geneva, Switzerland, Standard ISO 8855:2011, 2011.
- [5] ISO, "Robots and Robotic Devices – Coordinate Systems and Motion Nomenclatures", International Organization for Standardization, Geneva, Switzerland, Standard ISO 9787:2013, Mai 2013.
- [6] PX4 Development Team, *Using Vision or Motion Capture Systems for Position Estimation*, [https://docs.px4.io/main/en/ros/external\\_position\\_estimation](https://docs.px4.io/main/en/ros/external_position_estimation), Zugriff am 31.05.2026, 2024.
- [7] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer International Publishing, 2022, ISBN: 9783030343729. DOI: 10.1007/978-3-030-34372-9.
- [8] B. Siciliano, L. Sciavicco, L. Villani und G. Oriolo, *Robotics*. Springer London, 2009, ISBN: 9781846286421. DOI: 10.1007/978-1-84628-642-1.
- [9] *Springer Handbook of Robotics*. Springer International Publishing, 2016, ISBN: 9783319325521. DOI: 10.1007/978-3-319-32552-1.
- [10] Q. Quan, *Introduction to Multicopter Design and Control*. Springer Singapore, 2017, ISBN: 9789811033827. DOI: 10.1007/978-981-10-3382-7.
- [11] G. Bradski und A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008, ISBN: 978-0596516130. Adresse: <https://learning.oreilly.com/library/view/learning-opencv/9780596516130/ch11s05.html>.
- [12] J. S. Dai, "Euler–Rodrigues formula variations, quaternion conjugation and intrinsic connections", *Mechanism and Machine Theory*, Jg. 92, S. 144–152, 2015, ISSN: 0094-114X. DOI: <https://doi.org/10.1016/j.mechmachtheory.2015.03.004>. Adresse: <https://www.sciencedirect.com/science/article/pii/S0094114X15000415>.
- [13] V. Capuano, L. Xu und J. Estrada Benavides, "Smartphone MEMS Accelerometer and Gyroscope Measurement Errors: Laboratory Testing and Analysis of the Effects on Positioning Performance", *Sensors*, Jg. 23, Nr. 17, S. 7609, 2023, ISSN: 1424-8220. DOI: 10.3390/s23177609.

- [14] C. Chen und X. Pan, "Deep Learning for Inertial Positioning: A Survey", *IEEE Transactions on Intelligent Transportation Systems*, 2024. DOI: 10.1109/TITS.2024.3381161. arXiv: 2303.03757 [cs.RO].
- [15] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems", *Transactions of the ASME – Journal of Basic Engineering*, Jg. 82, Nr. 1, S. 35–45, 1960. DOI: 10.1115/1.3662552.
- [16] Y. Kim und H. Bang, "Introduction to Kalman Filter and Its Applications", in *Introduction and Implementations of the Kalman Filter*, F. Govaers, Hrsg., London: IntechOpen, 2018, Kap. 2. DOI: 10.5772/intechopen.80600. Adresse: <https://doi.org/10.5772/intechopen.80600>.
- [17] J. A. Mendoza-Mendoza, V. J. Gonzalez-Villela, G. Sepulveda-Cervantes, M. Mendez-Martinez und H. Sossa-Azuela, *Advanced Robotic Vehicles Programming: An Ardupilot and Pixhawk Approach*. Apress, 2020, ISBN: 9781484255315. DOI: 10.1007/978-1-4842-5531-5.
- [18] PX4 Development Team, *Holybro Pixhawk 6X*, [https://docs.px4.io/v1.16/en/flight\\_controller/pixhawk6x](https://docs.px4.io/v1.16/en/flight_controller/pixhawk6x), Abgerufen am 04.06.2026.
- [19] N. Y. Ko, W. Youn, I. H. Choi, G. Song und T. S. Kim, "Features of Invariant Extended Kalman Filter Applied to Unmanned Aerial Vehicle Navigation", *Sensors*, Jg. 18, Nr. 9, S. 2855, Aug. 2018, ISSN: 1424-8220. DOI: 10.3390/s18092855.

# Abbildungsverzeichnis

|     |                                                                       |   |
|-----|-----------------------------------------------------------------------|---|
| 2.1 | Vergleich der Koordinatensysteme [3] . . . . .                        | 3 |
| 2.2 | Roll, Pitch und Yaw am Beispiel Holybro X650 [3] . . . . .            | 4 |
| 2.3 | Komponenten des Pixhawk 6X: FMU-Modul und Baseboard V2A [3] . . . . . | 9 |

# Tabellenverzeichnis

2.1 Sensorbestückung des Pixhawk 6X [18] . . . . . 10

# Quellcodeverzeichnis

## 10 Anhang

Der Anhang enthält ergänzende Informationen, die für das Verständnis der Arbeit relevant sind, aber nicht direkt in den Haupttext integriert werden konnten. Dazu gehören beispielsweise ausführliche Tabellen, zusätzliche Abbildungen oder Quellcode, der für die Implementierung der beschriebenen Methoden verwendet wurde.